



Module 5 : Transformers & LLMs

Dr. Priya R L, Mrs. Sunita Suralkar

Faculty Incharge for NCMPE64 (Natural Language Processing & Generative AI)

Department of Computer Engineering

VES Institute of Technology, Mumbai

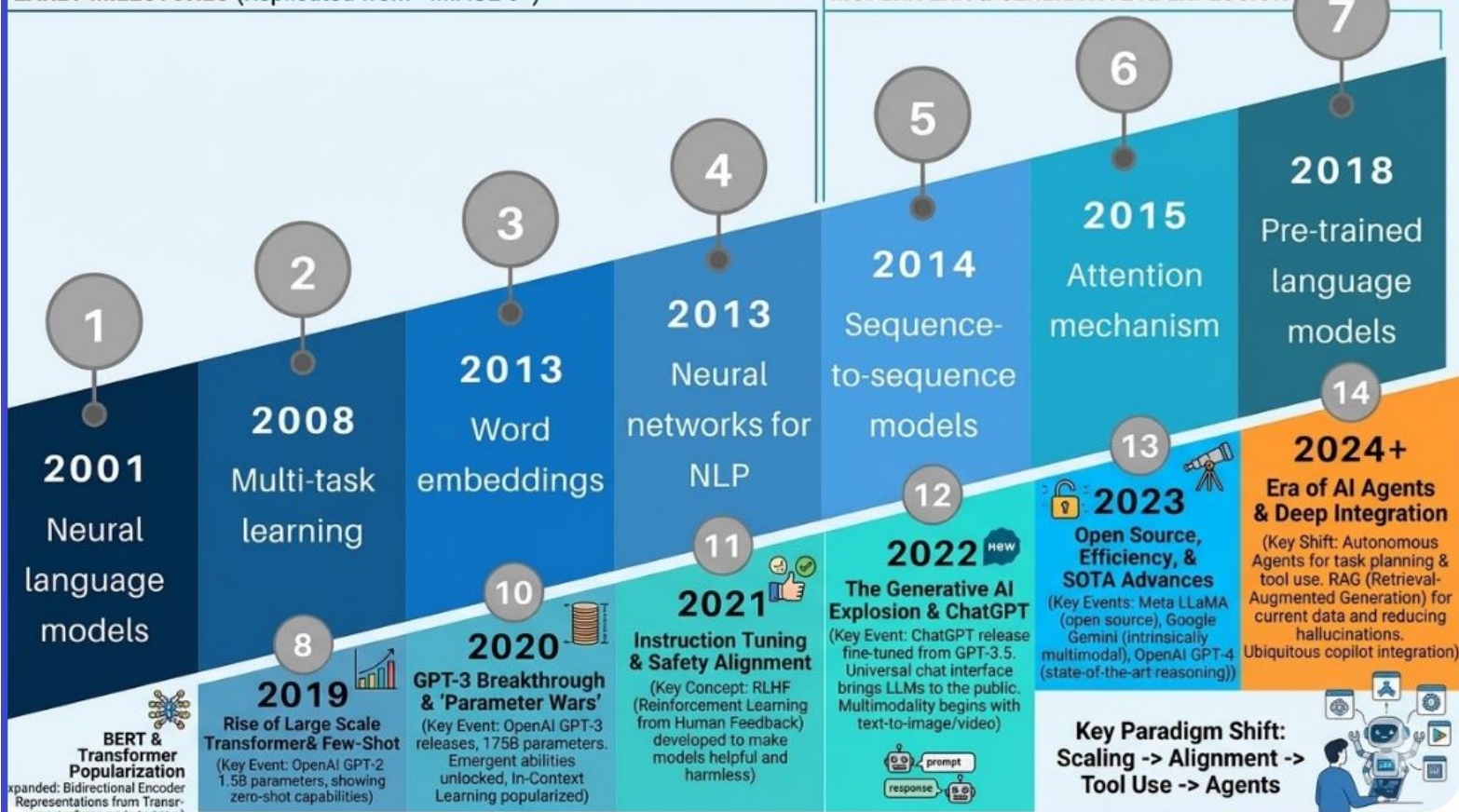
Agenda - Part : I

- Transformer Architecture and
- Attention Mechanism,
- Popular LLMs: BERT,
- GPT-4,
- RoBERTa,
- Meta LLaMA2,
- Google PaLM2

THE EVOLUTION OF NATURAL LANGUAGE PROCESSING: A DEFINITIVE TIMELINE

EARLY MILESTONES

MODERN ERA & GENERATIVE AI EXPLOSION

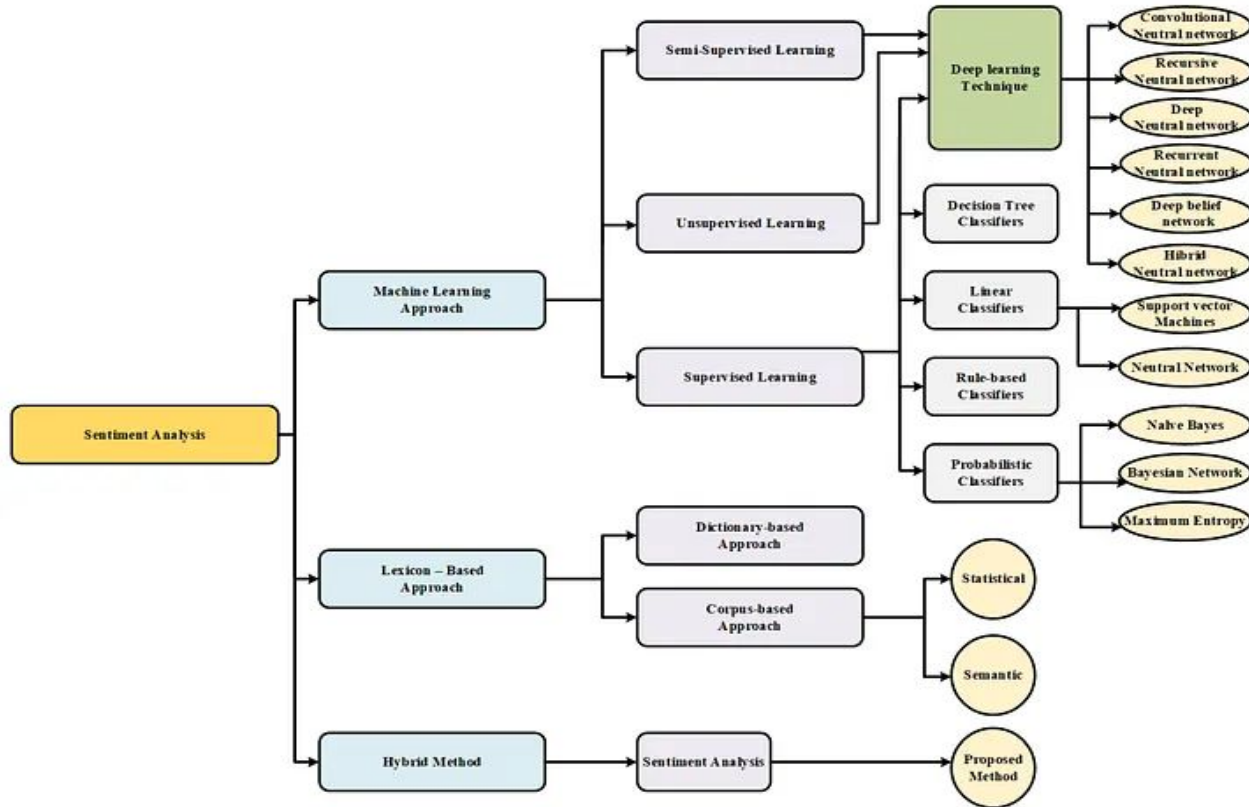




	Feature	Statistical(N-gram) Language Models	Neural Language Models
1	Representation	Uses discrete representations . Words are treated as unique IDs; the model doesn't know "cat" and "dog" are both animals.	Uses dense vectors (embeddings) . Words are mapped to a continuous space where similar words have similar values.
2	Context Window	Fixed and limited. Usually only looks at the previous 2–5 words (N-1). It cannot remember the beginning of a long sentence.	Flexible and long-range. Architectures like Transformers can process thousands of words simultaneously to maintain context.
3	Data Sparsity	High. If a specific sequence of words wasn't in the training data, the model gives it a 0% probability (requires "smoothing" to fix).	Low. Because it uses embeddings, it can predict likely words even if it hasn't seen that exact sequence before by using "neighboring" concepts.
4	Generalization	Poor. It struggles with synonyms or paraphrasing because it looks for exact word matches.	Excellent. It generalizes well across different writing styles and understands that "He is happy" is similar to "He is joyful."
5	Computational Cost	Low. Very fast to train and run; can often be handled by a standard laptop in seconds.	High. Requires massive computational power (GPUs/TPUs) and significant time to train.
6	Model Size	Scales poorly. As the context window (N) increases, the size of the probability table grows exponentially, making it impractical.	Scales efficiently. The number of parameters grows, but the model remains a compact mathematical function compared to a massive raw table.

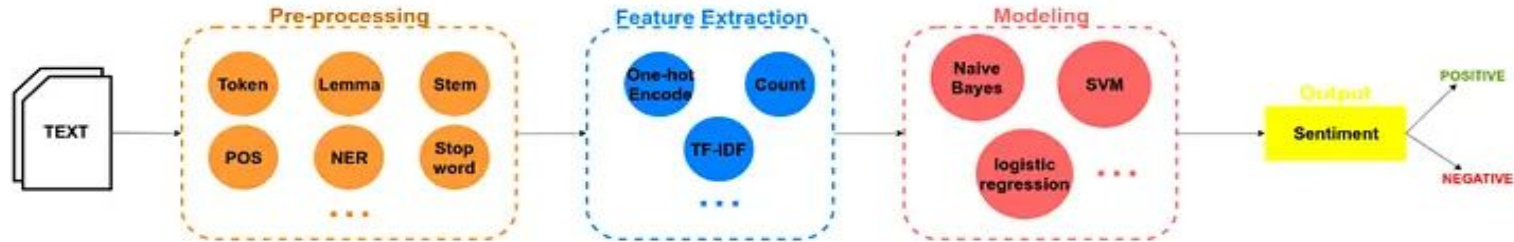
NCMPE64 : Natural Language Processing and Generative AI

Case Study: Sentiment Analysis

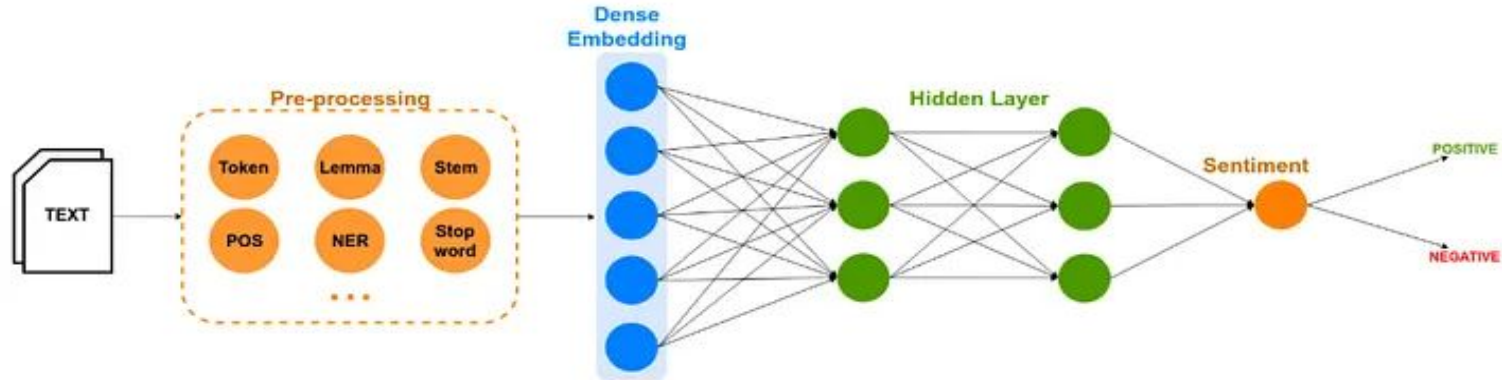


Case Study: Sentiment Analysis (ML vs DL)

Machine Learning



Deep Learning

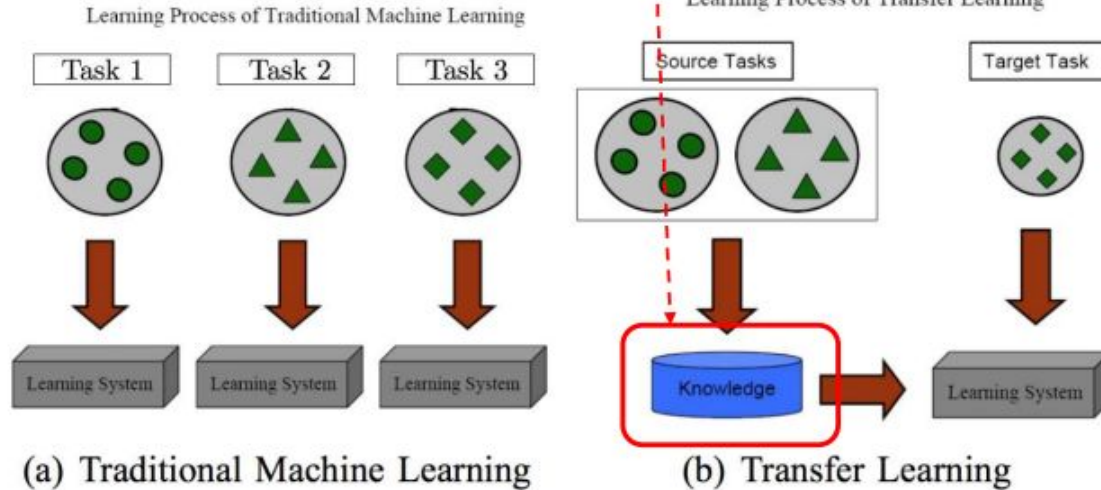


Pre-trained Language Representations

❖ How can we take advantage of distributed word representation?

➤ Transfer Learning

❖ What is Transfer Learning?

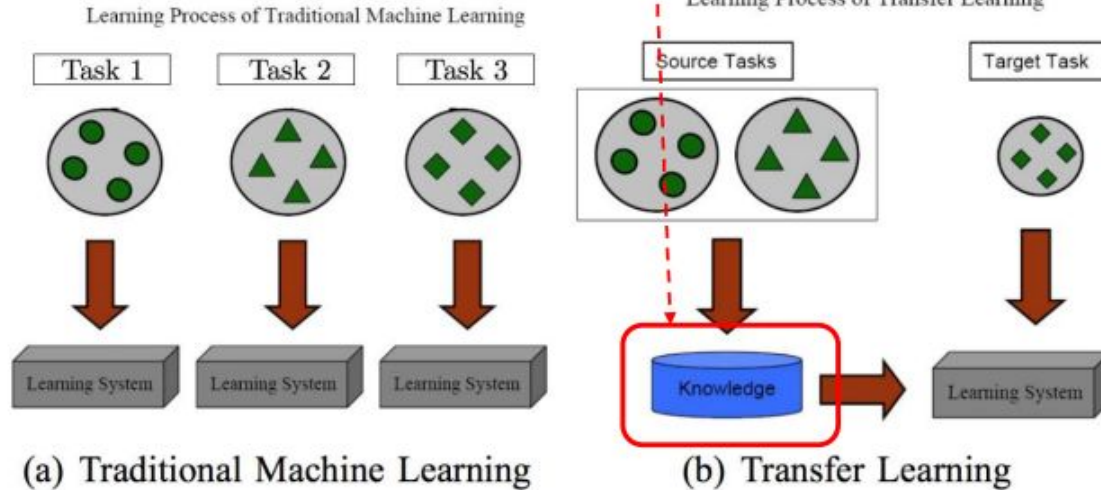


Pre-trained Language Representations

❖ How can we take advantage of distributed word representation?

➤ Transfer Learning

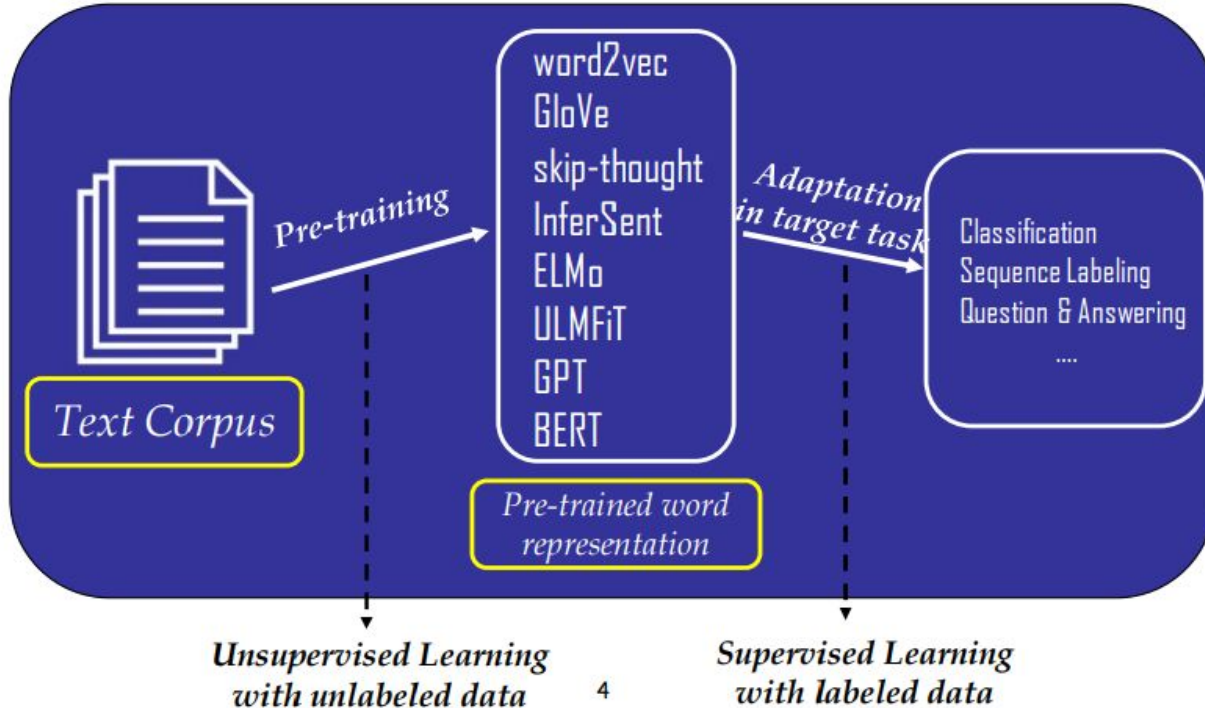
❖ What is Transfer Learning?



Pre-trained Language Representations

❖ Transfer Learning using word representations

➤ Pre-trained Word Representation

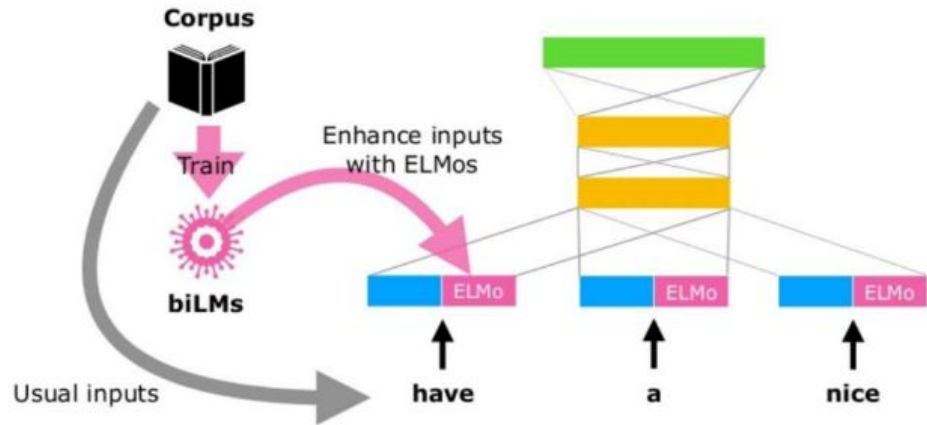


Pre-trained Language Representations

- **Two kinds of Pre-trained Language Representation:**
 - Feature-based Approach
 - Fine-tuning Approach

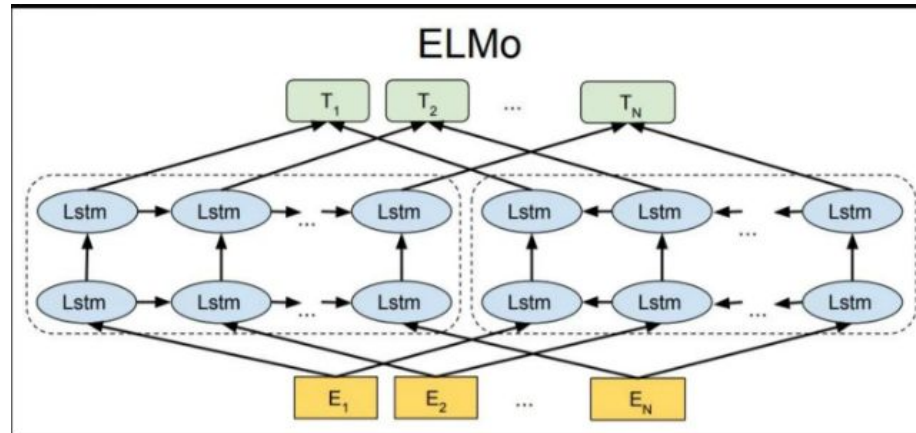
❖ Feature-based approach

- Use task-specific architectures that include the pre-trained representations as additional features
 - Learned representations are used as features in a downstream model
- ELMo



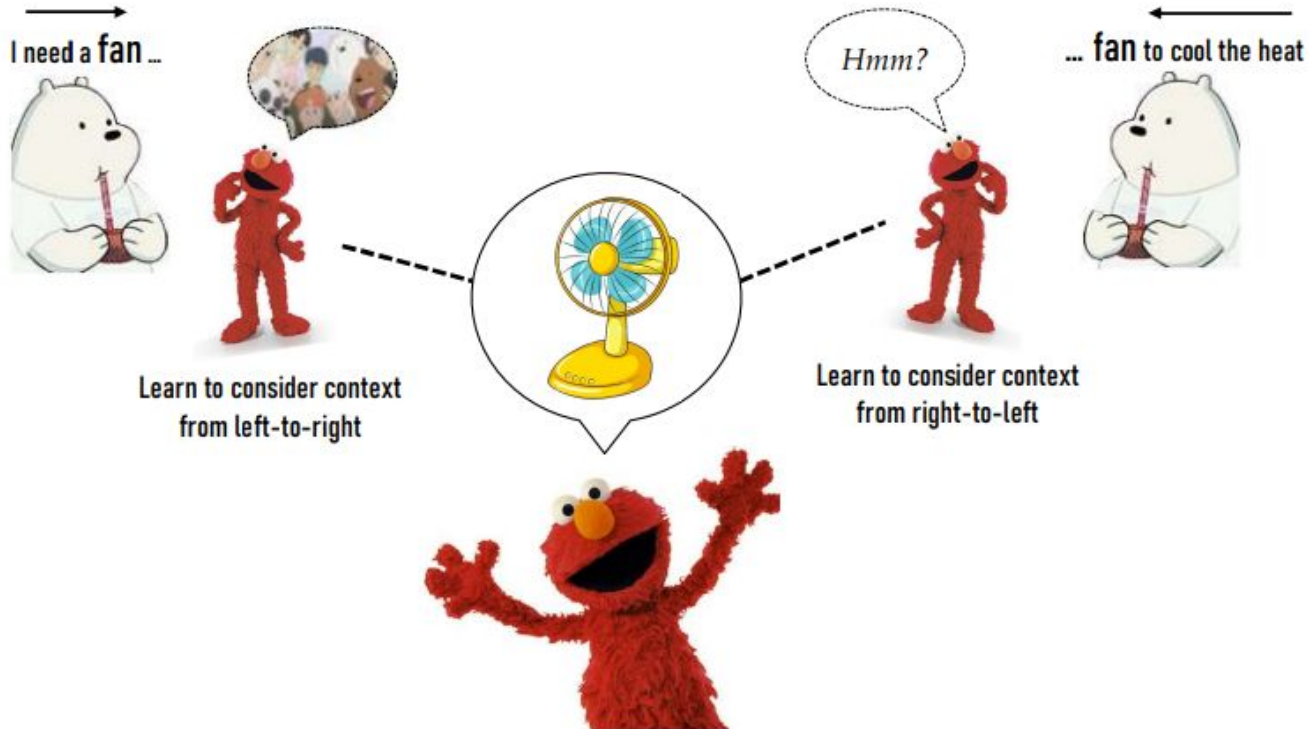
ELMo (Embeddings from Language Models)[2018]

- ELMo is a **deep contextualized word representation** that models both complex characteristics of word use (e.g., syntax and semantics) and how these uses vary across linguistic contexts (e.g., to model polysemy).
- ELMo generates **context-sensitive embeddings** using **deep bi-directional LSTM models**. It provides different embeddings for words based on their context. Contextual LM - Two-Way, Two-Layered



ELMo - Feature Based Approach

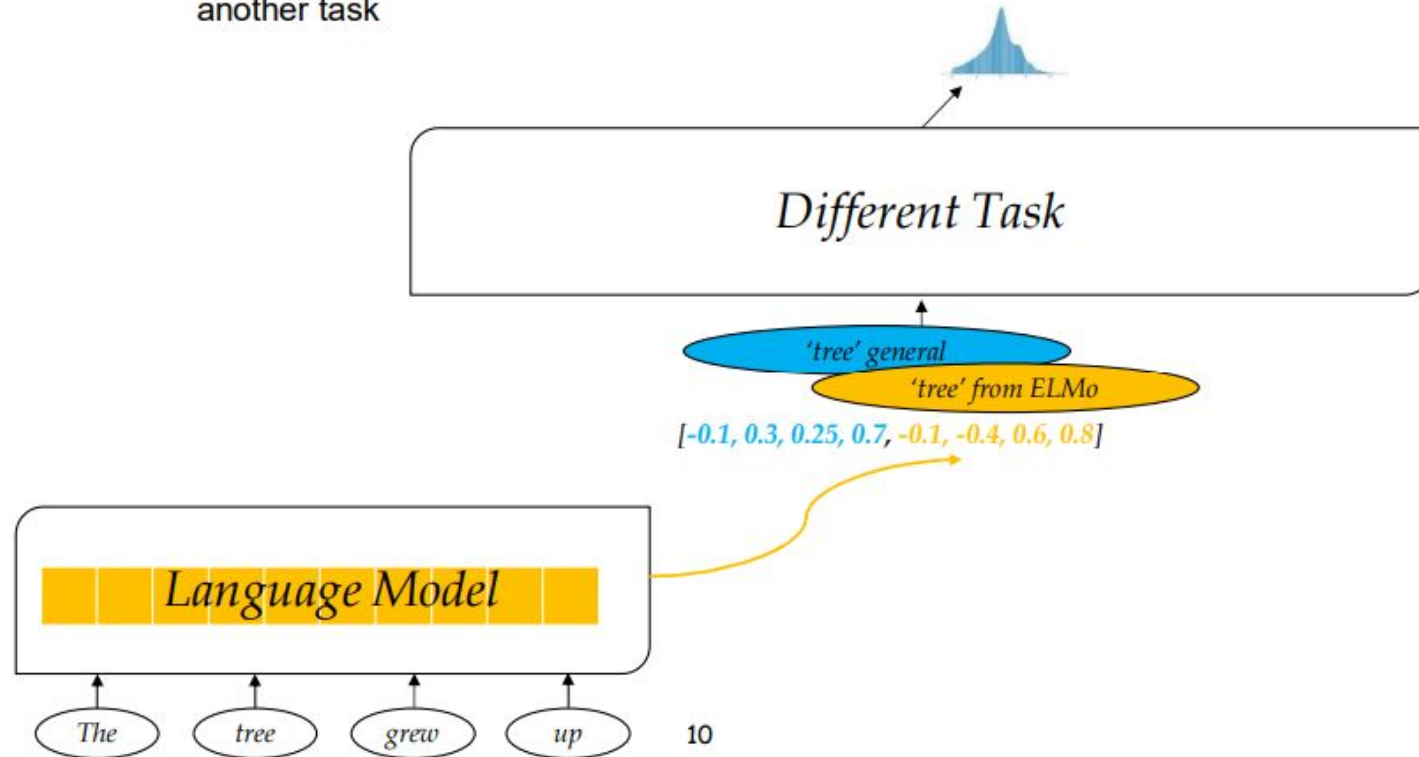
- Let the words be represented according to the context !!!



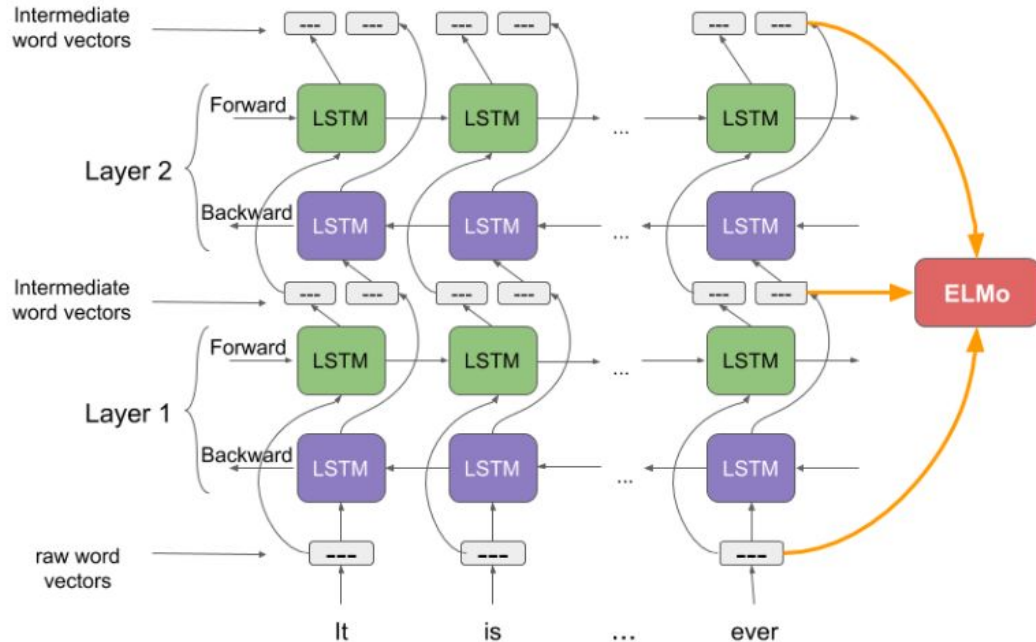
❖ ELMo (Peters et al. 2018) (Cont'd)

➤ Trained task-specifically

- Learned parameters(weights) from language model are used as a **feature** for another task



ELMo Inferences



- ELMo word vectors are computed on top of a **two-layer bidirectional language model (biLM)**. This biLM model has two layers stacked together. Each layer has 2 passes — forward pass and backward pass
- Uses a **character-level convolutional neural network (CNN)** to represent words of a text string into raw word vectors
- As the **input to the biLM is computed from characters rather than words**, it captures the inner structure of the word.

Key Features : ELMo

- **Contextualized Representations:** ELMo embeddings vary depending on the context in which a word appears. Different from traditional embeddings like Word2Vec or GloVe, which provide a single representation for each word regardless of context.
- **Deep Representations:** ELMo uses deep bidirectional LSTM networks to generate embeddings, capturing complex syntactic and semantic information.
- **Layer-wise Representations:** ELMo combines representations from all the layers of the biLSTM, which allows it to capture different types of information at different layers.
- **Pros:** Contextualized Embeddings, Bi-directional Context.
- **Cons:** Complexity and Size, Fixed context window.

Bidirectional Encoder Representations from Transformers - BERT

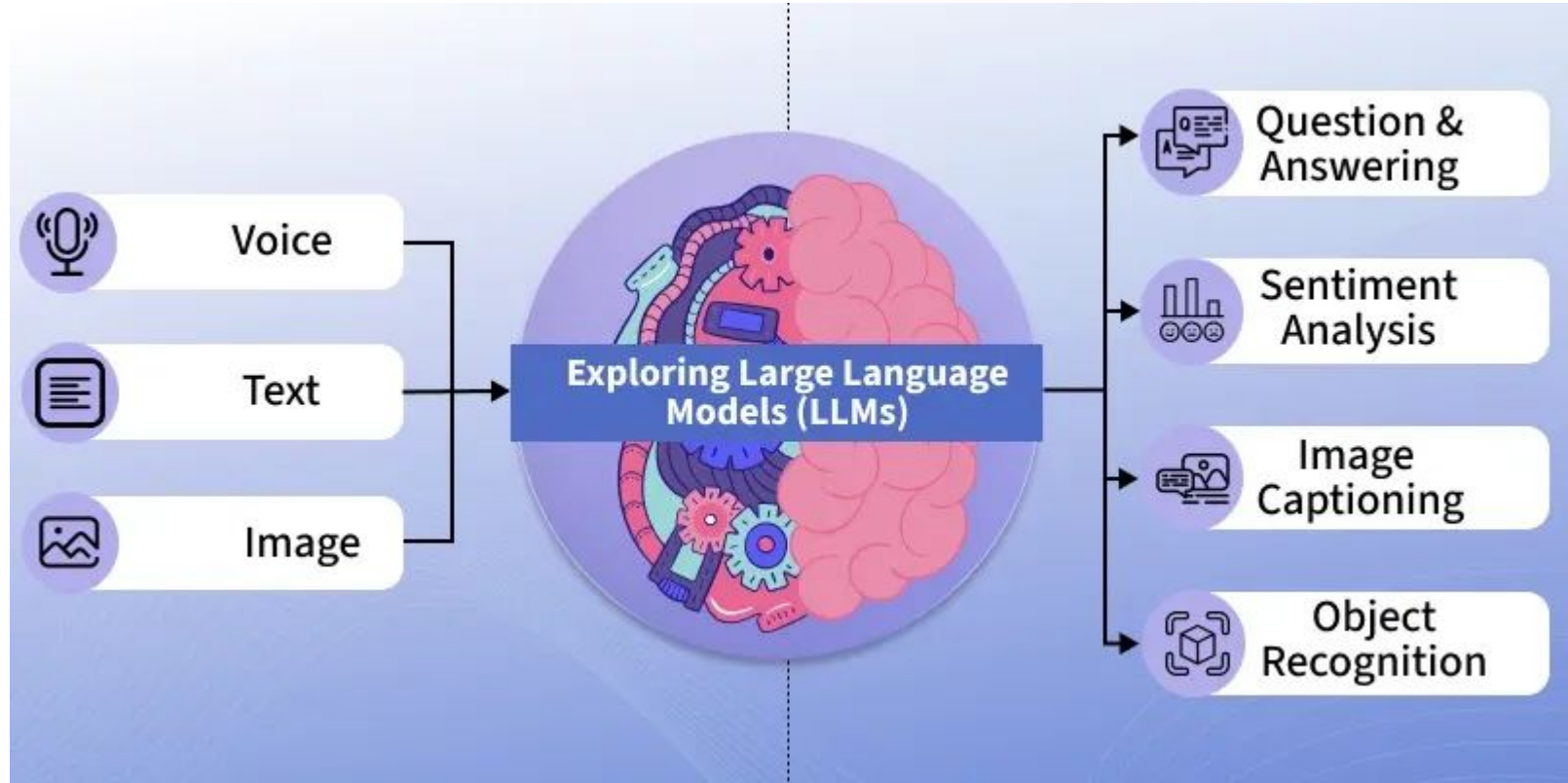
- BERT is a transformer-based model that generates context-aware embeddings by considering the entire sentence bidirectionally (i.e., both left-to-right and right-to-left).
- Unlike traditional word embeddings like Word2Vec or GloVe, which generate a single representation for each word, BERT produces different embeddings for each word based on its context.
- **Pros:** Pre-training and Fine-tuning, Contextualized Embeddings, Bi-directional Context.
- **Cons:** Complexity and Size, Inference Time.

What is a Large Language Model (LLM)

Large Language Models (LLMs) are advanced AI systems built on deep neural networks designed to process, understand and **generate human-like text**.

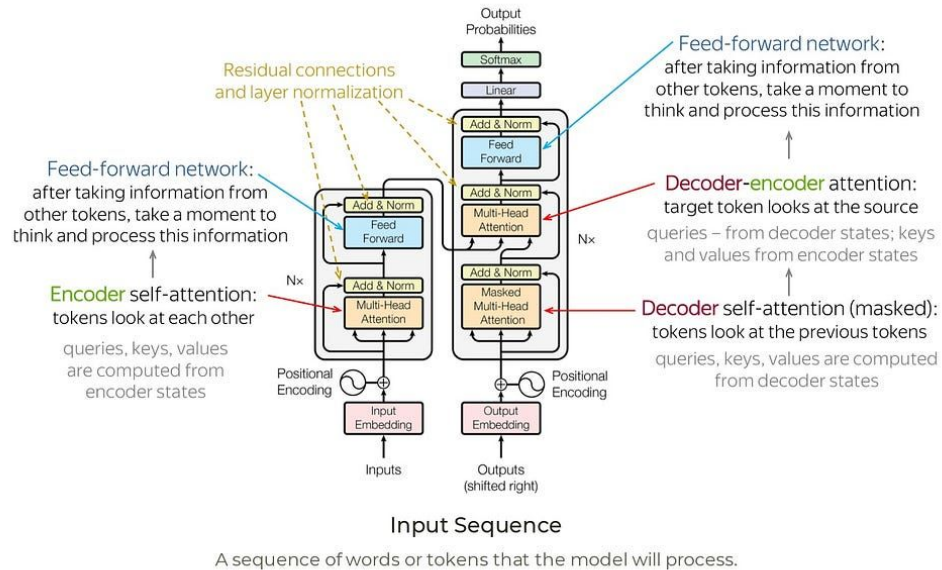
- LLMs **learn patterns, grammar and context from text** and can answer questions, write content, translate languages and many more.
- By using massive datasets and billions of parameters, LLMs have **transformed the way humans interact with technology**.
- Modern LLMs include **ChatGPT (OpenAI), Google Gemini, Anthropic Claude**, etc.

What is a Large Language Model (LLM)



Architecture of Large Language Model (LLM)

How Transformers Work: A Step-by-Step Breakdown





Working of LLM

A standard Transformer has two main parts:

Encoder

- Reads and understands the input text
- Converts words into meaningful representations
- An encoder converts input text into an intermediate representation. An encoder is an enormous neural net.

Decoder

- Generates output text (used in tasks like translation or text generation)
- A decoder converts that intermediate representation into useful text. A decoder is also an enormous neural net.
- **In LLMs:** Models like GPT use **only the decoder**, Models like BERT use **only the encoder**

Working of LLM

Every text-generative Transformer consists of these **three key components**:

1. **Embedding**: Text input is divided into **smaller units called tokens**, which can be words or subwords. These **tokens are converted into numerical vectors** called embeddings, which capture the semantic meaning of words.
2. **Transformer Block** is the **fundamental building block of the model** that processes and transforms the input data. Each block includes:
 - **Attention Mechanism**, the core component of the Transformer block. It allows tokens to communicate with other tokens, **capturing contextual information and relationships between words**.

Working of LLM (Cont'd..)

- **MLP (Multilayer Perceptron) Layer**, a feed-forward network that operates on each token independently. While the goal of the attention layer is to route information between tokens, the goal of the MLP is to refine each token's representation.

3. Output Probabilities: The final **linear and softmax layers transform the processed embeddings into probabilities**, enabling the model to make predictions about the next token in a sequence.

Working of LLM (Cont'd..)

- **Input Prompt: “Data visualization empowers users to”**
- It transforms the text into a numerical representation that the model can work with. To convert a prompt into embedding, we need to
 - 1) **tokenize** the input,
 - 2) obtain **token embeddings**,
 - 3) add **positional information**, and finally
 - 4) add up token and position encodings to get the **final embedding**

Eg.: GPT-2's vocabulary has **50,257 unique tokens**. GPT-2 (small) represents each token in the vocabulary as a **768-dimensional vector**; the dimension of the vector depends on the model. These embedding vectors are stored in a matrix of shape **(50,257, 768)**, containing approximately **39 million parameters**!

Working of LLM (Cont'd..)

Prompt:

Data visualization empowers users to ...



Working of LLM (Cont'd..) - Transformer Block

Multi-Head Self-Attention

- The self-attention mechanism enables the model to capture relationships among tokens in a sequence, so that each token's representation is influenced by the others.
- Multiple attention heads allow the model to consider these relationships from different perspectives;
- for example, one head may capture short-range syntactic links while another tracks broader semantic context.

Working of LLM (Cont'd..) - Transformer Block

Step 1: Query, Key, and Value Matrices

Each token's embedding vector is transformed into three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**. These vectors are derived by multiplying the input embedding matrix with learned weight matrices for **Q**, **K**, and **V**. Here's a web search analogy to help us build some intuition behind these matrices:

- **Query (Q)** is the **search text you type in the search engine bar**. This is the token you want to *"Learn more about Agentic AI"*.
- **Key (K)** is the **title of each web page in the search result window**. It represents the possible tokens the query can attend to.



Working of LLM (Cont'd..) - Transformer Block

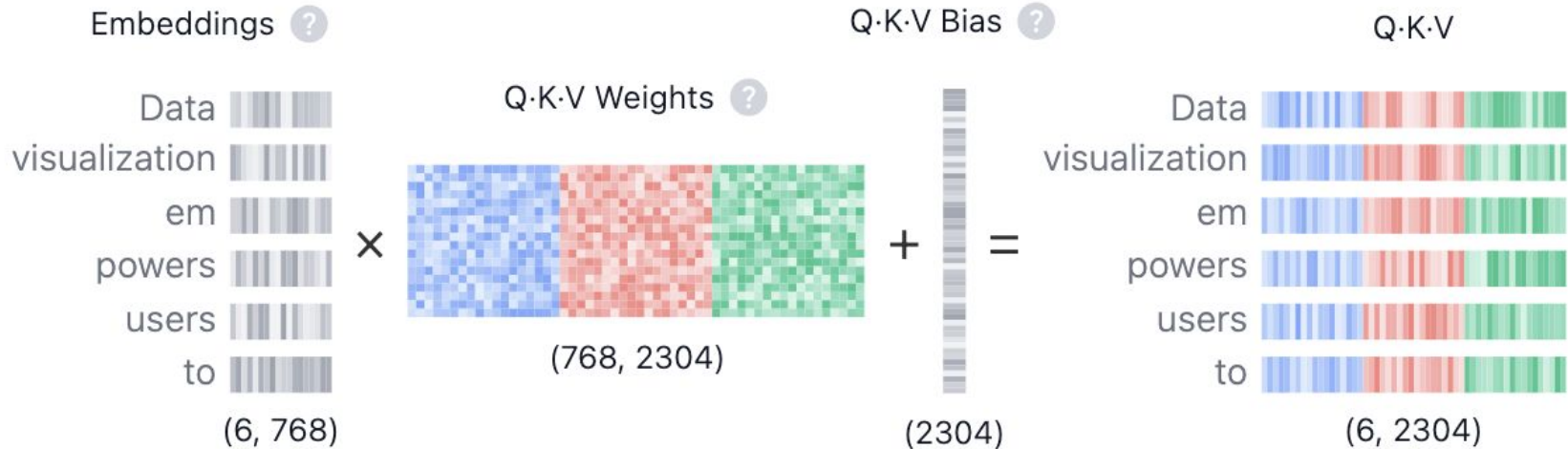
Step 1: Query, Key, and Value Matrices

- **Value (V)** is the **actual content of web pages shown**. Once we matched the appropriate search term (Query) with the relevant results (Key), we want to get the content (Value) of the most relevant pages.

By using these QKV values, the model can calculate attention scores, which determine how much focus each token should receive when generating predictions.

Working of LLM (Cont'd..) - Transformer Block

Step 1: Query, Key, and Value Matrices



Working of LLM (Cont'd..) - Transformer Block

Step 2: Multi-Head Splitting

- **Query**, **key**, and **Value** vectors are split into multiple heads—in GPT-2 (small)'s case, into **12 heads**.
- Each head processes a segment of the embeddings independently, capturing **different syntactic and semantic relationships**.
- This design facilitates parallel learning of **diverse linguistic features**, enhancing the model's representational power.



Working of LLM (Cont'd..) - Transformer Block

Step 3: Masked Self-Attention

- In each head, we perform masked self-attention calculations. This mechanism allows the model to generate sequences by focusing on relevant parts of the input while preventing access to future tokens.
- **Dot Product:** The dot product of Query and Key matrices determines the attention score, producing a square matrix that reflects the relationship between all input tokens.

Working of LLM (Cont'd..) - Transformer Block

Step 3: Masked Self-Attention

- In Transformer attention (specifically **self-attention**), we compute:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \text{mask} \right) V$$

Scaling (Why divide by $\sqrt{d_k}$)? The raw attention score is: QK^T

Problem: If the dimension d_k is large:

- Dot product values become **very large**
- This pushes softmax into extreme values (very peaky)
- Gradients become **very small** → **slow learning**

Solution: Scaling - We divide by $\sqrt{d_k}$: This keeps values in a stable range.

Working of LLM (Cont'd..) - Transformer Block

Step 3: Masked Self-Attention

- Why Masking? **To Prevent Cheating**
- **Problem in language models** : When predicting a sequence like:
 - **I love Natural Language Processing**

While predicting "**love**", the model **should NOT** see "**natuiral**" or "**language**" or "**Processing**"

Solution: Look-ahead mask (causal mask) - We create a matrix like:

$$\begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Working of LLM (Cont'd..) - Transformer Block

Step 3: Masked Self-Attention

- Upper triangle = **future tokens**
- These are set to $-\infty$
- **Why $-\infty$?** Because after softmax:

$$e^{-\infty} = 0$$

- So future tokens get **zero attention**

Working of LLM (Cont'd..) - Transformer Block

Step 3: Masked Self-Attention

- **Scaling · Mask:** The attention scores are scaled and a mask is applied to the upper triangle of the attention matrix to prevent the model from accessing future tokens, setting these values to negative infinity. The model needs to learn how to predict the next token without “peeking” into the future.
- **Softmax · Dropout:** After masking and scaling, the attention scores are converted into probabilities by the softmax operation, then optionally regularized with dropout. Each row of the matrix sums to one and indicates the relevance of every other token to the left of it.

Working of LLM (Cont'd..) - Transformer Block

Step 4: Output and Concatenation

- The model uses the **masked self-attention scores** and **multiplies them with the Value matrix** to **get the final output of the self-attention mechanism**.
- GPT-2 has 12 self-attention heads, each capturing different relationships between tokens.
- The **outputs of these heads are concatenated and passed through a linear projection**.

Working of LLM (Cont'd..) - Output Prob

- After the input has been processed through all **Transformer blocks**, the output is passed through the final linear layer to prepare it for **token prediction**.
- This layer projects the final representations into a 50,257 dimensional space, where **every token in the vocabulary has a corresponding value** called **logit**.
- Any token can be the next word, so this **process allows us to simply rank these tokens by their likelihood of being that next word**.
- We then apply the softmax function to convert the **logits into a probability distribution** that sums to one.
- This will allow us to sample the next token based on its likelihood.

Working of LLM (Cont'd..) - Output Prob

Temperature ? Sampling ? **Top-k** Top-p

0.8 k5

Tokens	Logits 	Scaled logits	Top-k	Softmax
visualize	-135.91	-169.89	-169.89	54.67%
create	-136.68	-170.85	-170.85	20.87%
see	-137.12	-171.40	-171.40	12.09%
make	-137.65	-172.06	-172.06	6.26%
easily	-137.67	-172.08	-172.08	6.11%
quickly	-137.72	-172.15	$-\infty$	0%
explore	-137.78	-172.23	$-\infty$	0%
find	-138.05	-172.56	$-\infty$	0%



Popular LLMs

Some of the most widely used LLMs include:

- **GPT-4 and GPT-4o (OpenAI):** Advanced multimodal reasoning and dialogue capabilities.
- **Gemini 1.5 (Google DeepMind):** Long-context reasoning, capable of handling 1M+ tokens.
- **Claude 3 (Anthropic):** Safety-focused, strong at reasoning and summarization.
- **LLaMA 3 (Meta):** Open-weight model, popular in research and startups.
- **Mistral 7B / Mixtral (Mistral AI):** Efficient open-source alternatives for developers.
- **BERT and RoBERTa (Google/Facebook):** Strong embedding models for NLP tasks.
- **mBERT and XLM-R:** Early multilingual LLMs.
- **BLOOM:** Large open-source multilingual model, collaboratively developed.

Attention Mechanism in ML

- It allows models to **focus on the most important parts of input data** when making predictions.
- It **assigns different weights to different elements** hence helping the model **prioritize relevant information** instead of treating all inputs equally.
- It forms the foundation of advanced models like Transformers and BERT and is widely used in Natural Language Processing (NLP) and Computer Vision.

Working: Attention Mechanism

Step 1: Input Encoding: The input sequence is first encoded using an encoder like RNN, LSTM, GRU or Transformer to generate hidden states representing the input context.

Step 2: Query, Key and Value Vectors: Each input is transformed into:

- **Query (Q):** Represents what we're looking for.
- **Key (K):** Represents what information each input contains.
- **Value (V):** Contains the actual information of each input.

These are linear transformations of the input embeddings.

Working: Attention Mechanism

Step 3: Key–Value Pair Creation: Each input is represented as a pair:

- **Key (K):** Represents the “address” or identifier of information.
- **Value (V):** Represents the actual content.

Step 4: Similarity Computation: The model computes similarity between the query and each key to determine relevance.

$$\text{Score}(s, i) = \begin{cases} h_s \cdot y_i & \text{(Dot Product)} \\ h_s^T W y_i & \text{(General)} \\ v^T \tanh(W[h_s; y_i]) & \text{(Concat)} \end{cases}$$

Where:

- h_s : Encoder hidden state at position s
- y_i : Decoder hidden state at position i
- W : Weight matrix
- v : Weight vector

Working: Attention Mechanism

Step 5: Attention Weights Calculation: The similarity scores are passed through a softmax function to convert them into attention weights:

$$\alpha(s, i) = \text{softmax}(\text{Score}(s, i))$$

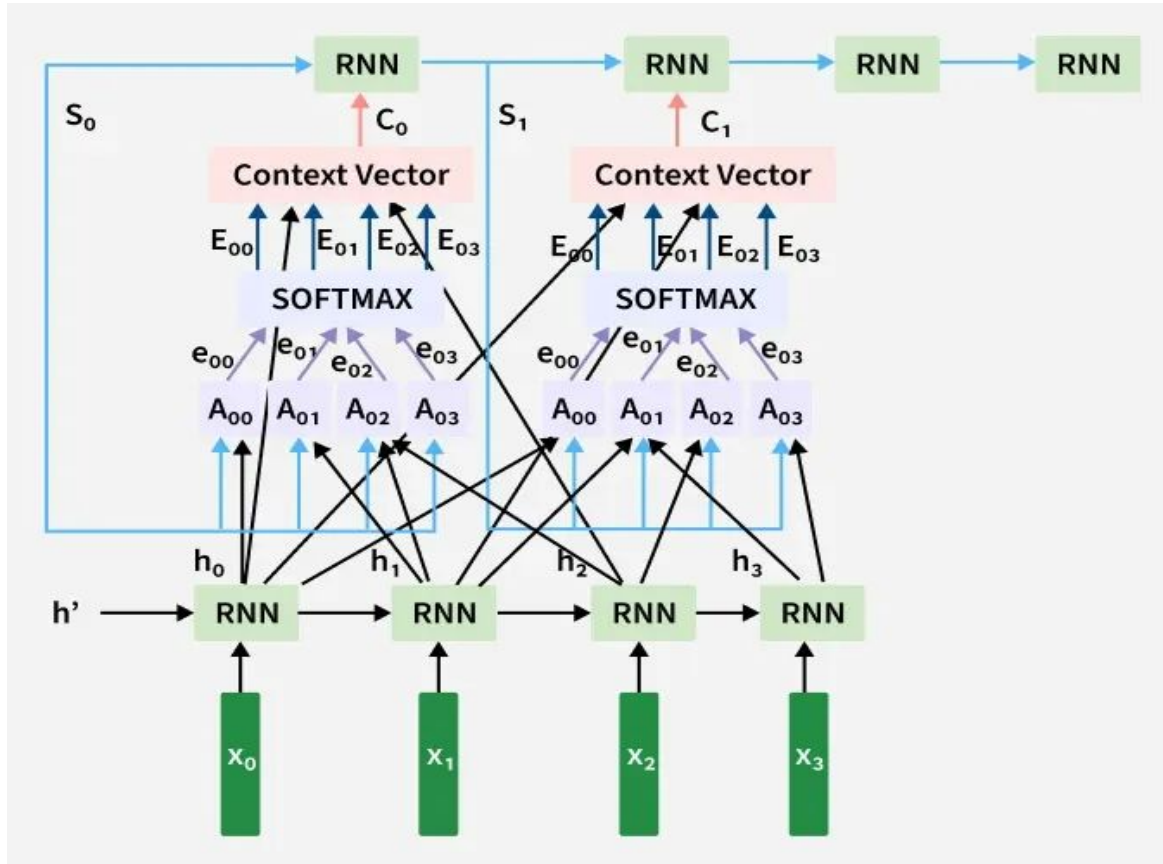
Step 6: Weighted Sum: The attention weights are used to compute a weighted sum of the value vectors:

$$c_t = \sum_{i=1}^{T_s} \alpha(s, i) h_i$$

Step 7: Context Vector: The context vector c_t summarizes the most relevant information from the input sequence and is fed to the decoder.

Step 8: Integration: The decoder uses both its own hidden state and the context vector to generate the next output token

Attention Mechanism Architecture



How Attention Mechanism Improves Traditional Deep Learning Models

Traditional deep learning models like RNNs, LSTMs and CNNs have limitations when handling long or complex dependencies. The attention mechanism enhances their effectiveness as follows:

- **RNNs/LSTMs:** These models compress the entire input into one vector, causing information loss over long sequences. Attention dynamically focuses on the relevant parts, resolving long-term dependency issues.
- **CNNs:** CNNs have fixed receptive fields and attention enables global dependencies, helping capture relationships beyond local patterns.
- **Seq2Seq Models:** Replace single context vectors with multiple dynamic ones, improving translation accuracy.
- **General Advantage:** Helps assign different importance weights to inputs, avoiding equal treatment of all tokens or features.

Applications

- **Machine Translation:** Focuses on relevant words while generating each output word.
- **Text Summarization:** Selects key information for concise summaries.
- **Image Captioning:** Attends to specific image regions to describe them accurately.
- **Sentiment Analysis & NER:** Highlights important words or entities in text.
- **Speech Recognition:** Focuses on critical audio frames for better transcription.



Advantages

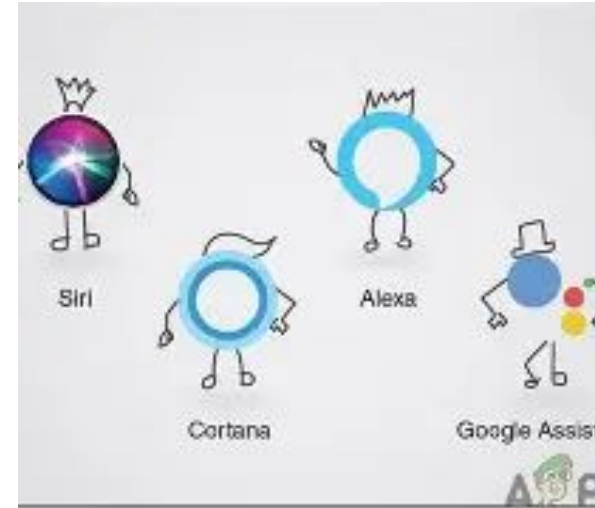
- Helps **models focus dynamically** on the most relevant information.
- **Solves long-term dependency issues** in sequential data.
- **Improves performance and interpretability** in NLP and Vision tasks.
- Enables **parallel computation** (in self-attention) unlike RNNs.
- **Enhances context understanding** in transformer-based models.

Limitations

- **Computationally expensive** for long sequences (especially self-attention).
- **Requires large memory** due to quadratic complexity.
- **Attention weights can be difficult to interpret in large models.**
- **Needs large datasets** for effective training.

Dialogue Systems and Chatbots

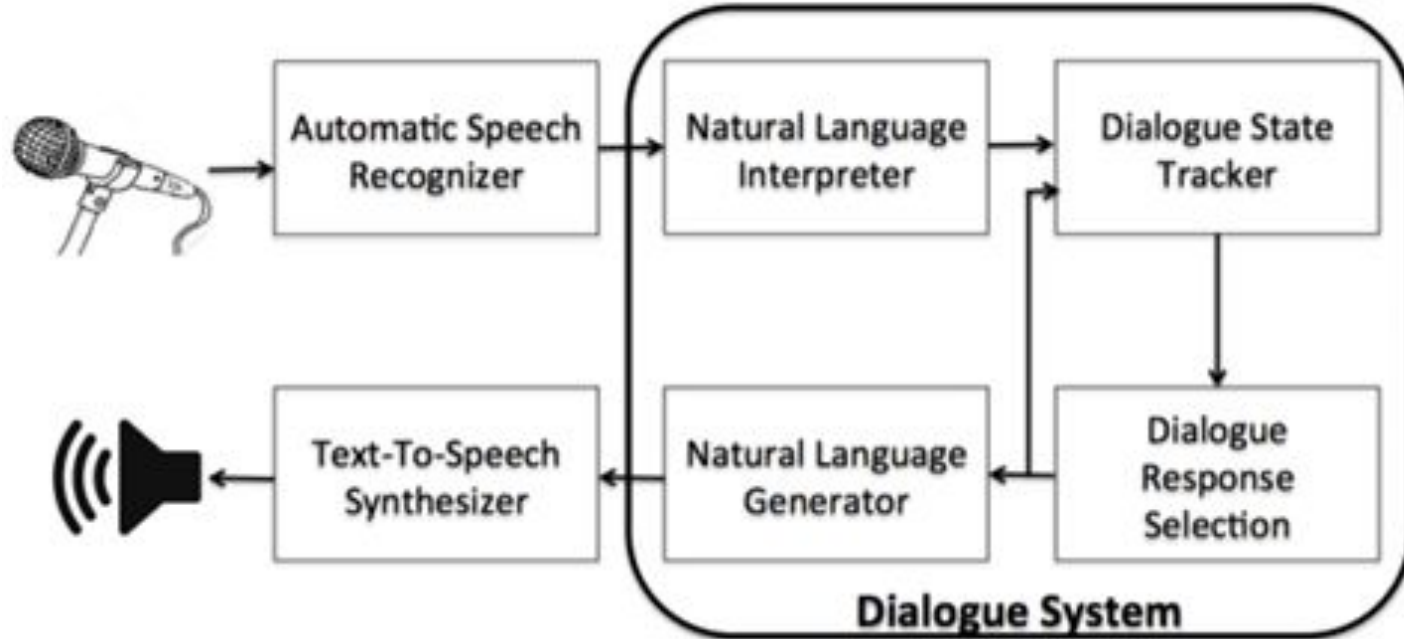
- Personal Assistants on phones or other devices
 - SIRI, Alexa, Cortana, Google Assistant
- Playing music, setting timers and clocks
- Chatting for fun
- Booking travel reservations
- Clinical uses for mental health



Dialogue Systems and Chatbots

- Two kind of conversational agents
 1. **Chatbots**- mimic informal human chatting- for fun, or even for therapy
 2. **(Task-based) Dialogue Agents**- interfaces to personal assistants- cars, robots, appliances- booking flights or restaurants

Dialogue Systems and Chatbots





Information Retrieval and RAG

- User has an information need
- And has some collection of documents
- User wants to find a relevant document
 - a document (or documents)
 - in the collection
 - that satisfy their need

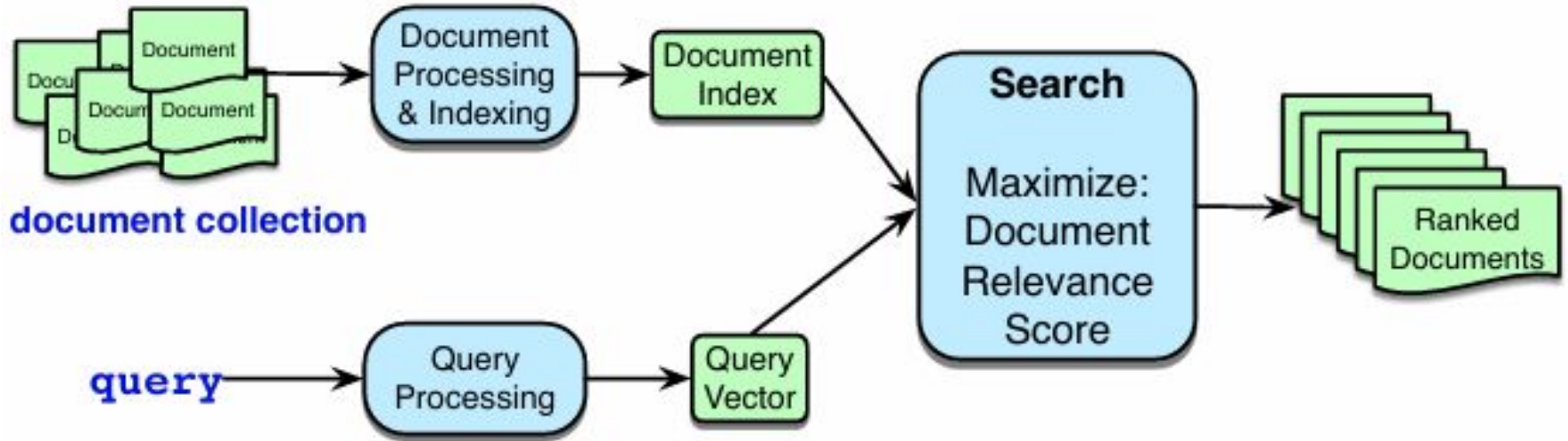
The vector space model of IR

1. Represent each document as a vector of counts of the words it contains.
2. Represent a query as a vector too
3. Return the document whose vector is most similar to the query vector

Ranked Retrieval

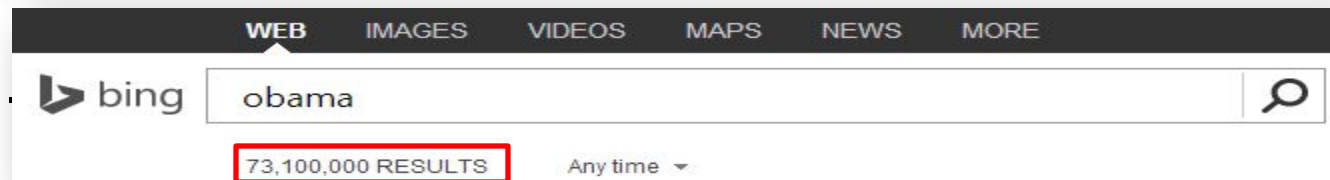
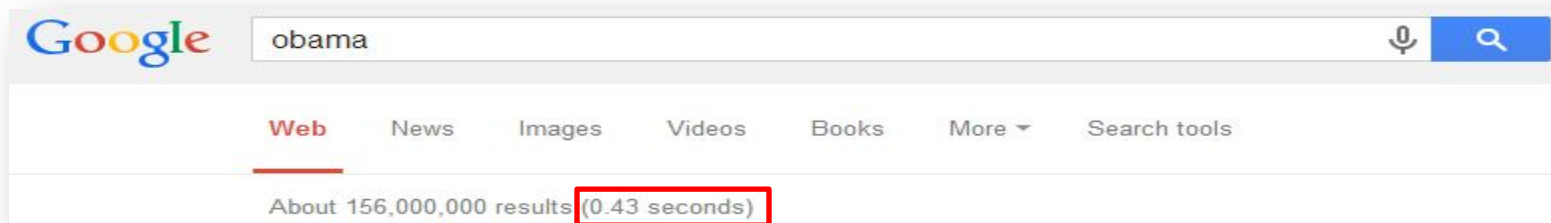
- The retriever returns top-k documents
- These are ranked
- We can show the user these, or some subset

Retrieval



Which search engine do you prefer: Bing or Google?

- What are your judging criteria?
 - How fast does it respond to your query?



Collection and Documents

The curse of the black pearl

Ship Jack Sparrow
Caribbean Turner
Elizabeth Gun Fight

Finding Nemo

Ocean Fish Nemo
Reef Animation

Tintin

Ocean Animation
Ship Haddock
Tintin

Titanic

Ship Rose Jack
Atlantic Ocean
England Sink

The Dark Knight

Bruce Wayne Batman
Joker Harvey Gordon
Gun Fight Crime

Skyfall

007 James Bond
MI6 Gun Fight

Silence of the Lambs

Hannibal Lector
FBI Crime Gun
Cannibal

The Ghost Ship

Ship Ghost Ocean
Death Horror

- Document: unit of retrieval
- Collection: the group of documents from which we retrieve
 - Also called *corpus* (a body of texts)

Boolean retrieval

The curse of the black pearl

Ship Captain Jack
Sparrow Caribbean
Elizabeth Gun Fight

Finding Nemo

Ocean Fish Nemo
Reef Animation

Tintin

Ocean Animation
Ship Captain
Haddock Tintin

Titanic

Ship Rose Jack
Atlantic Ocean
England Sink
Captain

The Dark Knight

Bruce Wayne Batman
Joker Harvey Gordon
Gun Fight Crime

Skyfall

007 James Bond
MI6 Gun Fight

Silence of the Lambs

Hannibal Lector
FBI Crime Gun
Cannibal

The Ghost Ship

Ship Ghost Ocean
Death Horror

- Find all documents containing a word w
- Find all documents containing a word w_1 but not containing the word w_2
- Queries in the form of any Boolean expression
- Query: **Jack**

Boolean retrieval

The curse of the black pearl

Ship Captain Jack
Sparrow Caribbean
Elizabeth Gun Fight

Finding Nemo

Ocean Fish Nemo
Reef Animation

Tintin

Ocean Animation
Ship Captain
Haddock Tintin

Titanic

Ship Rose Jack
Atlantic Ocean
England Sink
Captain

The Dark Knight

Bruce Wayne Batman
Joker Harvey Gordon
Gun Fight Crime

Skyfall

007 James Bond
MI6 Gun Fight

Silence of the Lambs

Hannibal Lector
FBI Crime Gun
Cannibal

The Ghost Ship

Ship Ghost Ocean
Death Horror

- Find all documents containing a word w
- Find all documents containing a word w_1 but not containing the word w_2
- More complicated Boolean queries
- Query: **Jack**

Term – document matrix

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- The entry $(w, d) = 1$ if and only if the word w is present in document d
- Terms are dimensions of this matrix (*units of index; we will discuss later*)
- Commonly called *term – document matrix*
- Term and word are not same, though often words are used as terms

Boolean retrieval

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- Query: Jack
- Results: 10010000

Boolean retrieval

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- Query: Captain AND Gun
- Results: 10110000 && 10001110 = 10000000

Query and relevant documents

- Query: given by user, represents the *information need*
 - Information need is the topic, conceptually what the user wants to know
 - Query is the representation of information need that the user conveys to the retrieval system
- Relevant document: a document that satisfies the information need, as perceived by the user
 - Merely matching the query terms does not mean a document is relevant
 - A relevant document must satisfy the actual information need

Precision and recall

- Precision: what fraction of the returned results are relevant?
 - Given a query q and a document d , need a judgment whether d is relevant for q
- Recall: what fraction of the relevant documents in the collection were returned by the system?
 - Given a query q , need the set D_q of all relevant documents that are relevant to q

What if the collection is “large”?

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- About 1 million documents (still not so large)
- About 500,000 distinct terms
- A term – document matrix of $500,000 \times 1$ million Boolean entries $\sim 500\text{GB}$

What if the collection is “large”?

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

Sparse matrix → inverted index

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1		1	1				1
Jack	1			1				
Bond						1		
Gun	1				1	1	1	
Ocean	1	1	1	1				1
Captain	1		1	1				
Batman					1			
Crime					1		1	

- In reality term – document matrices are very sparse
- Most terms are NOT present in most documents
- For every term, store only the documents where the term is present

Sparse matrix → inverted index

	1 Black pearl	2 Finding Nemo	3 Tintin	4 Titanic	5 Dark Knight	6 Skyfall	7 Silence of lambs	8 Ghost ship
Ship	1		1	1				1
Jack	1			1				
Bond						1		
Gun	1				1	1	1	
Ocean	1	1	1	1				1
Captain	1		1	1				
Batman					1			
Crime					1		1	

- Represent documents by document IDs

Sparse matrix → inverted index

1	2	3	4	5	6	7	8
Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship

Ship	→
Jack	
Bond	
Gun	
Ocean	
Captain	
Batman	
Crime	

Sparse matrix → inverted index

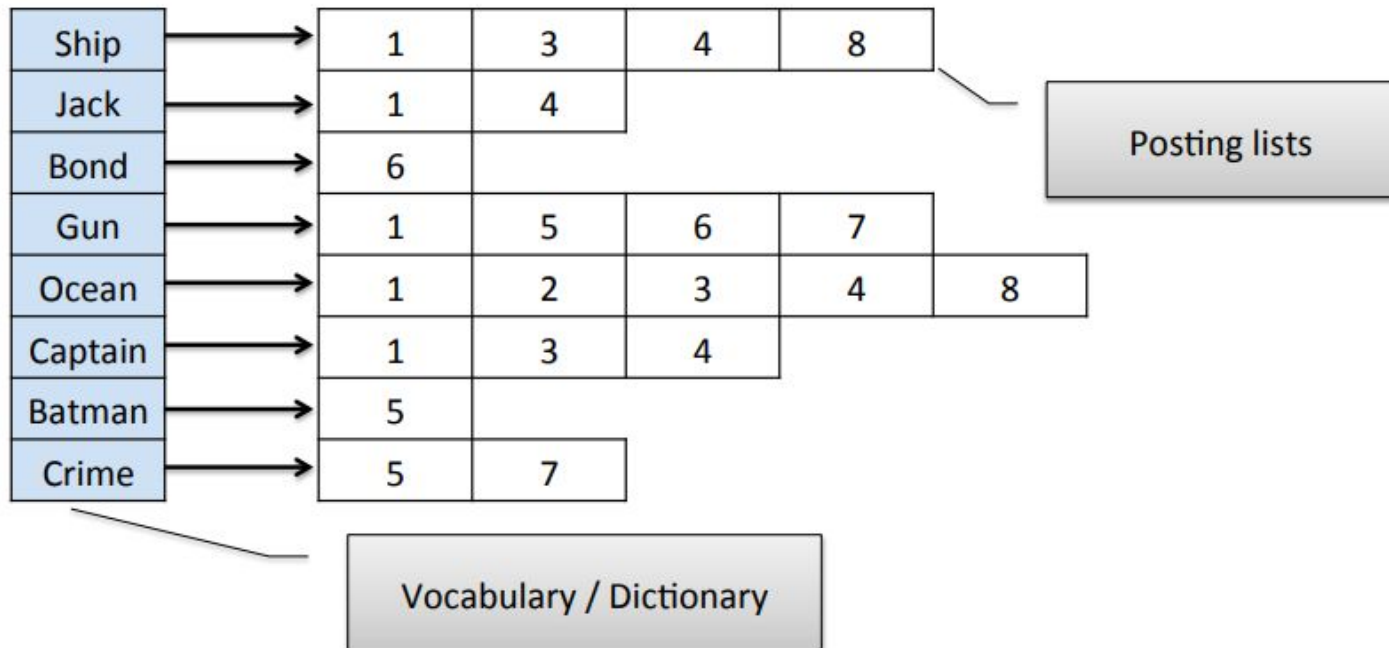
1	2	3	4	5	6	7	8
Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship

Ship	→	1	3	4	8
Jack					
Bond					
Gun					
Ocean					
Captain					
Batman					
Crime					

Sparse matrix → inverted index

1	2	3	4	5	6	7	8
Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skytall	Silence of lambs	Ghost ship

Inverted Index



Creating an inverted index

1 Curse of the Black Pearl
 Ship Captain Jack Sparrow
 Caribbean Elizabeth Gun
 Fight

2 Finding Nemo
 Ocean Fish
 Nemo Reef
 Animation

3 Tintin
 Ocean Animation
 Ship Captain
 Haddock Tintin

4 Titanic
 Ship Rose Jack Atlantic
 Ocean England Sink
 Captain

5 The Dark Knight
 Bruce Wayne Batman
 Joker Harvey Gordon Gun
 Fight Crime

6 Skyfall
 007 James Bond
 MI6 Gun Fight

7 Force of the Lamb
 Hannibal Lector FBI
 Crime Gun Cannibal

8 The Ghost Ship
 Ship Ghost Ocean Death
 Horror

Term	docId
Ship	1
Captain	1
Jack	1
...	...
Ship	3
Tintin	3
...	...
Jack	4
...	...

Sort by
term
➔

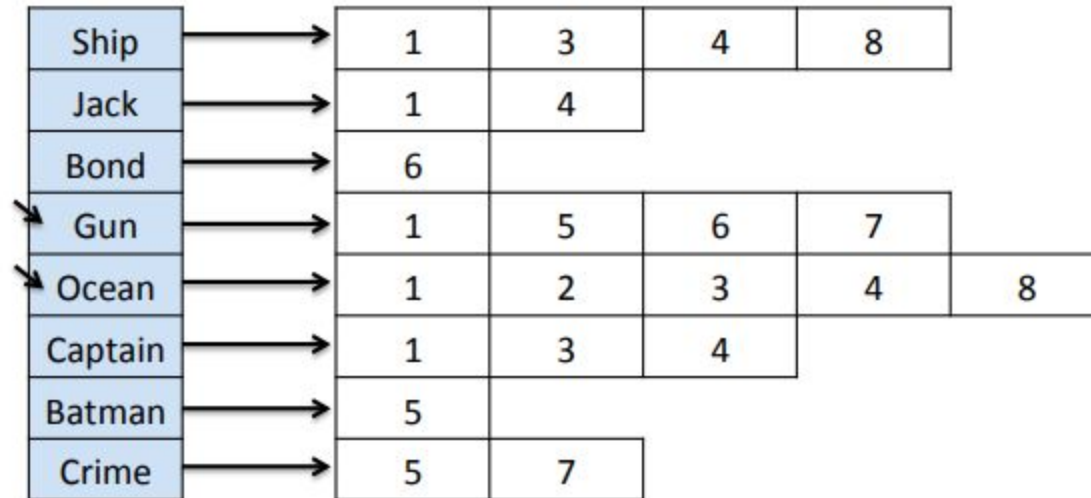
Term	docId
...	...
Captain	1
...	...
Jack	1
Jack	4
...	...
Ship	1
Ship	3
...	...

- For each document, write out pairs (term, docid)
- Sort by term, then group

Term	docId	docId	docId
Captain	1	...	
Jack	1	4	...
Ship	1	3	...
...	...		

Boolean query processing

Query: Gun OR Ocean



Boolean query processing

Query: Gun OR Ocean

Ship	→	1	3	4	8	
Jack	→	1	4			
Bond	→	6				
Gun	↘	1	5	6	7	
Ocean	↘	1	2	3	4	8
Captain	→	1	3	4		
Batman	→	5				
Crime	→	5	7			

Results:

1

Boolean query processing

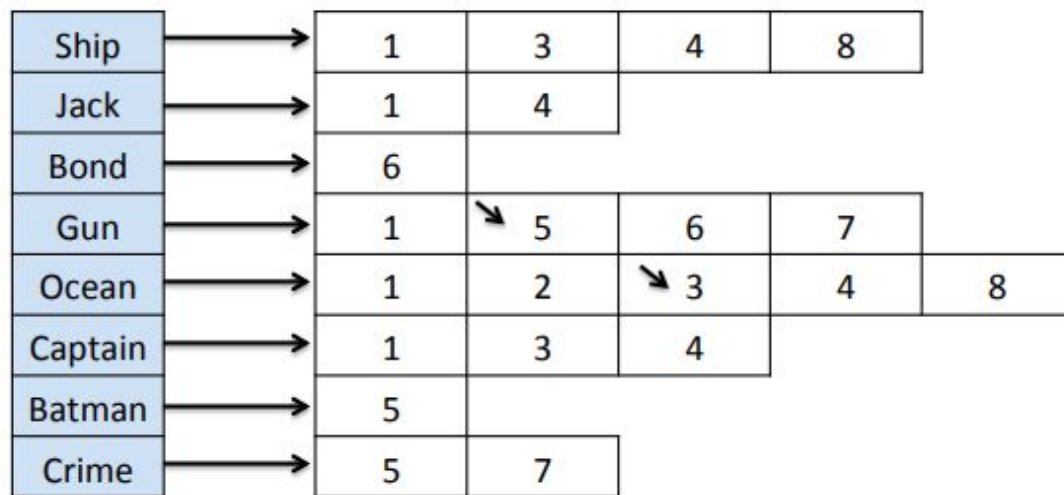
Query: Gun OR Ocean

Ship	→	1	3	4	8	
Jack	→	1	4			
Bond	→	6				
Gun	→	1	↘ 5	6	7	
Ocean	→	1	↘ 2	3	4	8
Captain	→	1	3	4		
Batman	→	5				
Crime	→	5	7			

Results: 1 2

Boolean query processing

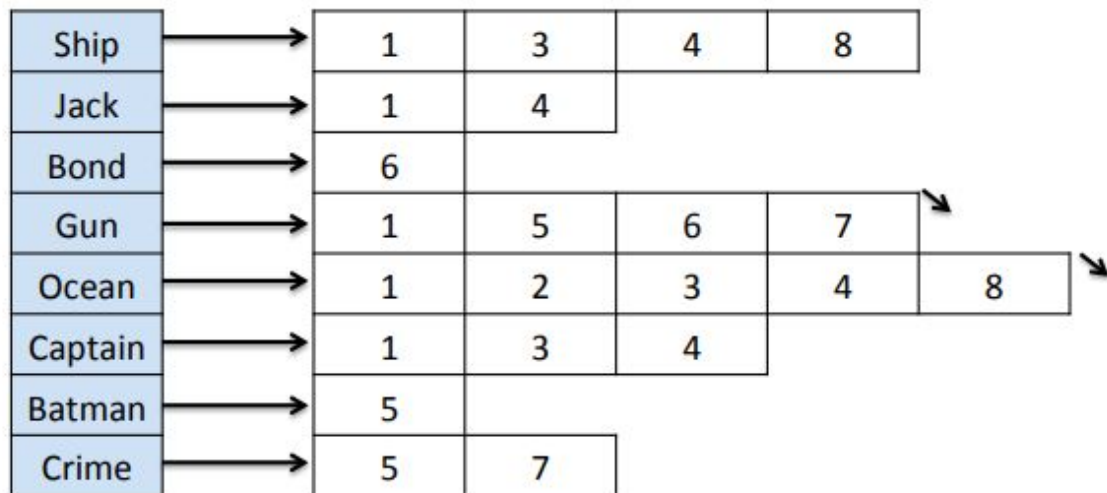
Query: Gun OR Ocean



Results: 1 2 3

Boolean query processing

Query: Gun OR Ocean

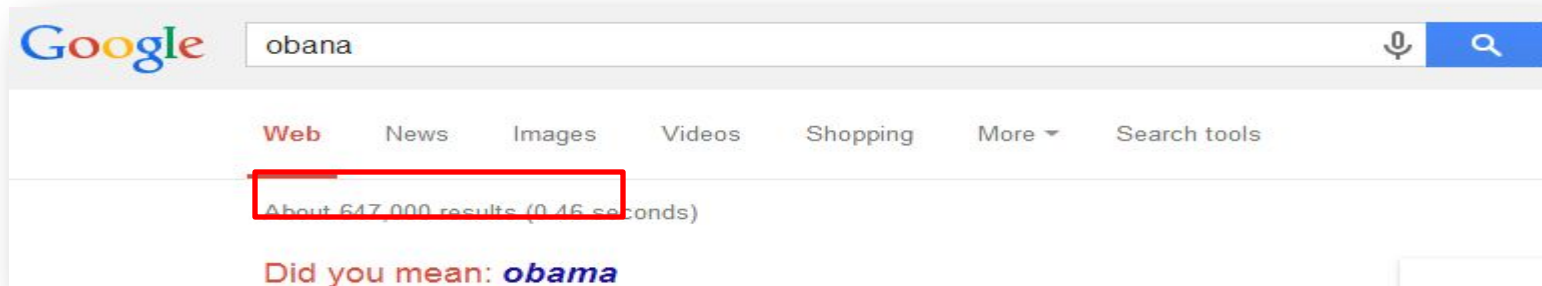


Results:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Which search engine do you prefer: Bing or Google?

- What are your judging criteria?
 - Can it correct my spelling errors?



- Can it suggest me good related queries?

Searches related to obama

sue obama	obama biograph
impeach obama	obama speech
obama approval rating	malia obama
obama jokes	obama scandal

Not just the web

Searching our email

Searching corporate documents

Searching personal medical records

Retrieval evaluation

- Evaluation criteria are all good, but not essential
 - Goal of any IR system
 - Satisfying users' information need
 - Core quality measure
 - *“how well a system meets the information needs of its users.” – wiki*
 - Unfortunately vague and hard to execute

Quantify the IR quality measure

- Information need
 - *“an individual or group's desire to locate and obtain information to satisfy a conscious or unconscious need” – wiki*
 - Reflected by user query

Quantify the IR quality measure

- Satisfaction
 - *“the opinion of the user about a specific computer application, which they use” – wiki*
 - Reflected by
 - Increased result clicks
 - Repeated/increased visits
 - Result relevance

Classical IR evaluation



- Cranfield experiments
 - Pioneer work and foundation in IR evaluation
 - Basic hypothesis
 - Retrieved documents' relevance is a good proxy of a system's utility in satisfying users' information need
 - Procedure
 - 1,398 abstracts of aerodynamics journal articles
 - 225 queries
 - Exhaustive relevance judgments of all (query, document) pairs

Classical IR evaluation

- Three key elements for IR evaluation
 1. A document collection
 2. A test suite of information needs, expressible as queries
 3. A set of relevance judgments, e.g., binary assessment of either *relevant* or *non-relevant* for each query-document pair

Evaluation metric

- To answer the questions
 - Is Google better than Bing?
 - Which ranking method is the most effective?
 - Shall we perform stemming or stopword removal?
- We need a quantifiable metric, by which we can compare different IR systems
 - As unranked retrieval sets
 - As ranked retrieval results

Evaluation of unranked retrieval sets

- In a Boolean retrieval system
 - Precision: fraction of retrieved documents that are relevant, i.e., $p(\text{relevant}|\text{retrieved})$
 - Recall: fraction of relevant documents that are retrieved, i.e., $p(\text{retrieved}|\text{relevant})$

	relevant	nonrelevant
retrieved	true positive (TP)	false positive (FP)
not retrieved	false negative (FN)	true negative (TN)

Precision:

$$P = \frac{TP}{TP + FP}$$

Recall:

$$R = \frac{TP}{TP + FN}$$

Evaluation of unranked retrieval sets

- Precision and recall trade off against each other
 - Precision decreases as the number of retrieved documents increases (unless in perfect ranking), while recall keeps increasing
 - These two metrics emphasize different perspectives of an IR system
 - Precision: prefers systems retrieving fewer documents, but highly relevant

Evaluation of ranked retrieval results

- Summarize the ranking performance with a single number
 - Binary relevance
 - Eleven-point interpolated average precision
 - Precision@K (P@K)
 - Mean Average Precision (MAP)
 - Mean Reciprocal Rank (MRR)
 - Multiple grades of relevance
 - Normalized Discounted Cumulative Gain (NDCG)

Precision@K

- Set a ranking position threshold K
- Ignores all documents ranked lower than K
- Compute precision in these top K retrieved documents

← Relevant
← Nonrelevant



– E.g.,

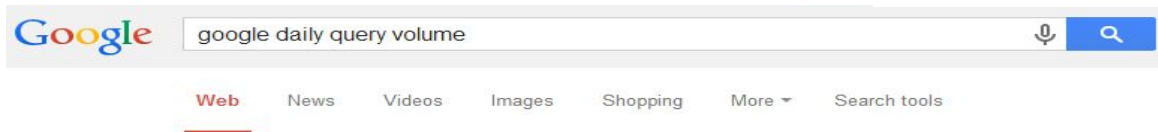
P@3 of 2/3

P@4 of 2/4

P@5 of 3/5

• In a similar fashion we have Recall@K

Beyond binary relevance



About 5,910,000 results (0.42 seconds)

Google Search Statistics - Internet Live Stats

www.internetlivestats.com/google-search-statistics/

Historical search volume, growth rate, and Google's share of global search market. ... launched, Google was already answering 3.5 million search queries daily.

Google Annual Search Statistics | Statistic Brain

www.statisticbrain.com/google-searches/

The first funding for Google was an August 1998 contribution of US\$100,000 from ... Year, Annual Number of Google Searches, Average Searches Per Day.

Insight into Google Search Query Numbers and What It ...

getstat.com/google-search-queries-the-numbers/

Jul 27, 2012 - Insights into the true meaning of Google Search Queries and the numbers behind it. ... Google has had an immense impact on how we operate in everyday ... Each month, the sheer volume of queries it answers continues to ...

How many search queries does Google serve worldwide ...

www.quora.com/How-many-search-queries-does-Google-serve-w...

Answer 1 of 8: This is latest data that Matt Cutts update yesterday - Google has seen more than 30 trillion URLs and crawls 20 billion pages a day. 3 billion...

Google Trends

www.google.com/trends/

50,000+ searches. Image Source - New York Daily News - David Wilson. 50,000+ searches. Image Source - NBCSports.com - Marilyn Burns. 20,000+ searches.

Google Trends - Wikipedia, the free encyclopedia

en.wikipedia.org/wiki/Google_Trends

Google Trends also allows the user to compare the volume of searches between ... the information provided by Google Trends daily; Hot Trends is updated hourly. ... Because the relative frequency of certain queries is highly correlated with the ...

P@6
MAP
MRR

Same P@6?!

Same MAP?!

Relevant

Nonrelevant

Excellent

Good

Fair

Fair

Bad

Bad

Ranked retrieval

The retriever returns top-k documents

These are ranked

We can show the user these, or some subset.

Two architectures for measuring query-doc relevance (= similarity)

Sparse retrieval

- ☐ represent query and doc as **vectors of word counts**
- ☐ weighted by tf-idf, BM25

Dense retrieval

- ☐ Use LLM to represent query and doc as **embeddings**

In both cases, similarity is usually the **cosine** between query and document representations

Information Retrieval in the LLM Age

If we have some information needs we don't need a document collection!

- Where is the Louvre Museum located?
- Where does the energy in a nuclear explosion come from?
- How to get a script l in latex?

Just prompt an LLM!

Information Retrieval in the LLM Age

But IR is still relevant for LLMs!

Retrieving relevant documents can still help solve problems that LLMs have with meeting information needs!

What are these problems?

LLMs Hallucinate

Hallucination: a response that is not faithful to the facts of the world.

In the legal domain LLMs were shown to hallucinate up to 88% of the time!

Dahl et al. (2024)

Can't use Proprietary Data

People need to ask questions about:

- personal email
- healthcare applications to medical records
- internal corporate documents
- legal documents discovery

These are (hopefully) not in the pretraining data of large language models!

Can't Handle Dynamic Data

LLMs can't answer questions about rapidly changing information like

- the sports match that happened last week
- the earthquake that happened this morning
- the latest Oscar winner

In general, data shifts over time!

Solution: RAG

Retrieval-Augmented Generation

1. Use IR to **retrieve** documents from some collection
2. Then use LLM to **generate** an answer conditioned on the documents

Two kinds of question answering datasets

Natural information-seeking questions

- Someone actually wanted to know the answer to this

Probing (testing) questions

- Exam-type questions for LLM evaluation